

HardF-SNN:

Hardware-Friendly Quantization for Spiking Neural Networks with Efficient Integer-Arithmetic-Only Inference

Introduction :

SNNs have been widely used for image recognition, audio recognition, object recognition etc. ANNs use MAC operations whereas SNNs use simple accumulate operations since spikes are binary. Research has been done on making SNNs accurate, but this has lead to higher complexity and thus higher sizes of SNNs being deployed which makes it difficult to use them in resource constrained devices

Thus, the paper solves this issue by 2 methods.(This is called model compression):

- Proportional shared scale quantisation
- Batch Normalisation

Quantisation :

Quantisation is reducing the parameters of a model from **high computation floating points** to easier **integers representation** (low bit width)

This not only saves memory since integers occupy less space than floating points, but also make computation easy with low precision arithmetic.

Present attempts on Quantisation :

Weights have been quantised successfully but the aspect of membrane potentials have been largely ignored.

This is because Membrane potentials are dynamic - they continuously accumulate values over time. Any quantization error compounds across timesteps, potentially causing incorrect spike behaviour.

Moreover, weights and MP are stored in SRAM, which requires frequent accessing to memory leading to lot of time delays.

Quantising MP also, has lead to accuracy degradation

Additionally, these quantised SNNs, have some dependency on FPUs since they do something called simulated quantisation (round values to integers but convert them again to FPUs, backward pass (gradient) is calculated through FPUs,

SIMULATED QUANTISATION :

Forward pass — Take the float weight (e.g. 0.73), fake-quantize it to integer and back ($0.73 \rightarrow 7 \rightarrow 0.7$), then use that 0.7 to compute the output. So the output is calculated using a *degraded float*, not a true integer.

Loss calculation Compare that degraded output to the true label. This comparison and error calculation is in **floating point**.

Backward pass - Gradients flow through in full floating point. The real weights (stored in float) get updated.

Deployment - Now you throw away the floats and replace everything with actual integers

These are incompatible with FPGAs since they allow only integer based computation.

Hence, HardF SNN made a baseline model having share scale quantisation along with batch normalisation folding. However, this suffered from performance degradation.

proportional shared-scale quantization, which better preserves the distribution of membrane potentials , something that the share scale quantisation failed to do.

$$\mathbf{X}_{int}^l = \left\lfloor \text{clip}\left(\frac{\mathbf{X}^l}{\alpha}, -2^{b-1}, 2^{b-1} - 1\right) \right\rfloor, \hat{\mathbf{X}}^l = \alpha \mathbf{X}_{int}^l. \quad (4)$$

This quantisation is done during training and the integer values are then stored for use during inference.

$$\begin{aligned} \mathbf{H}^l[t] &= \mathbf{U}_{int}^l[t-1] + \mathbf{W}_{int}^l \mathbf{S}^{l-1}[t], \\ \tilde{\mathbf{U}}^l[t] &= \alpha_{\mathbf{q}} \mathbf{H}^l[t], \\ \mathbf{S}^l[t] &= \begin{cases} 1, & \text{if } \mathbf{H}^l[t] \geq \theta / \alpha_{\mathbf{q}} \\ 0, & \text{otherwise} \end{cases}. \end{aligned}$$

Batch Normalisation :

This is a technique used to keep the distribution of the inputs in every layer same since if BN is not done, mean, variance and other factors of the input change after every computation and thus the process becomes slow (smaller steps taken for accurate result when inputs vary highly)
Thus, more epochs needed.

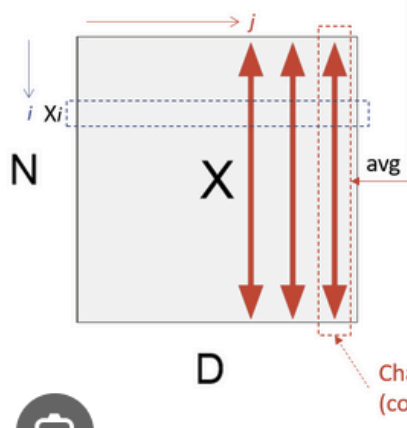
Thus batch normalization is done before quantising the weights

Feature Normalization uses the mean of entire data of a feature, but cannot fit in the memory, hence mean per batch

Batch Normalization

[Ioffe and Szegedy, 2015]

Input: $x : N \times D$



Average of a feature (j) in each batch X

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean,
shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel var,
shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

Normalized x ,
Shape is $N \times D$

Channel is Feature
(column j in X)

Each feature (height, age) is independent, but image pixels are within the same range (e.g 0-255) and related. Why not use average of all data in such cases?

$$W_{fold} = \frac{\gamma W}{\sqrt{EMA(\sigma^2) + \epsilon}}.$$

γ are floating-point linear affine factors, $EMA(\sigma^2)$ denotes the exponential moving average estimates of the variance of the convolution outputs across batches, and ϵ is a small constant for numerical stability.

Proportional Shared Scaling Quantisation :

Due to the shared scaling factor, the membrane potentials are compressed into the same dynamic range as the weights. This causes substantial clipping of their long-tailed distribution, thereby significantly reducing the dynamic range of membrane potentials

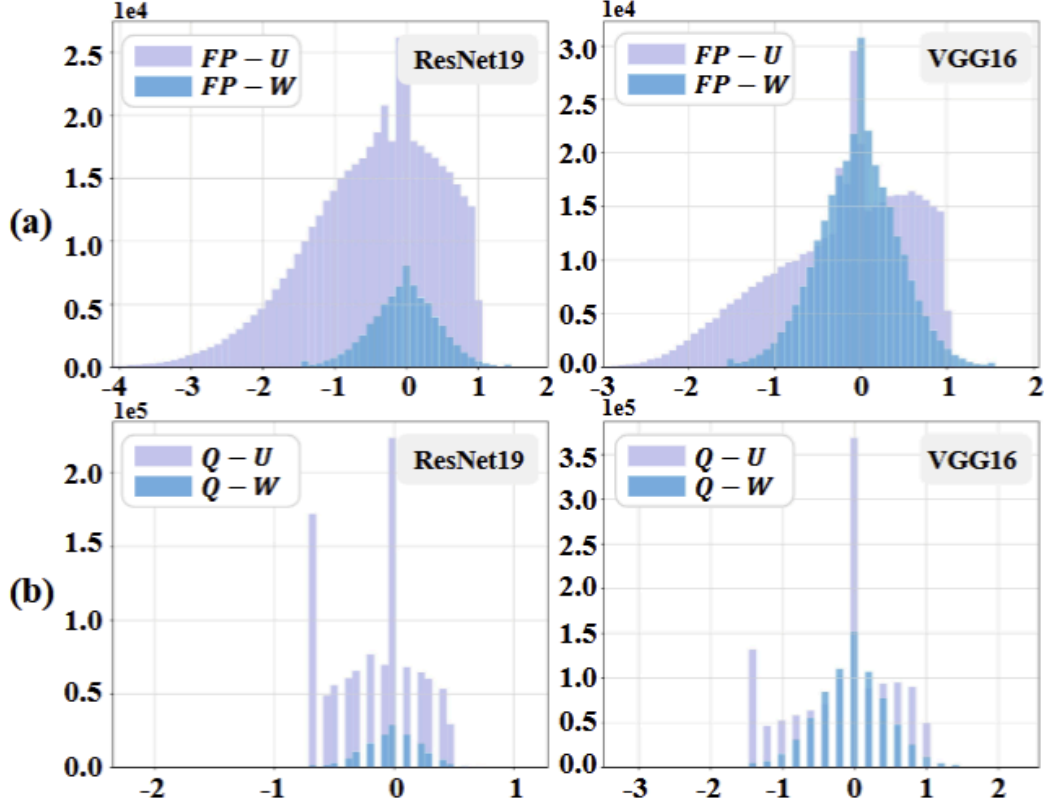


Figure 1: (a) Distinct distributions of full-precision weights and membrane potentials. (b) Distributions of weights and membrane potentials after 4-bit shared-scale quantization.

$$\mathbf{H}^l[t] = \eta \mathbf{U}_{int}^l[t-1] + \mathbf{W}_{int}^l \mathbf{S}^{l-1}[t],$$

$$\tilde{\mathbf{U}}^l[t] = \alpha_w \mathbf{H}^l[t],$$

$$\mathbf{S}^l[t] = \begin{cases} 1, & \text{if } \mathbf{H}^l[t] \geq \theta / \alpha_w \\ 0, & \text{otherwise} \end{cases}.$$

Batch Normalisation :

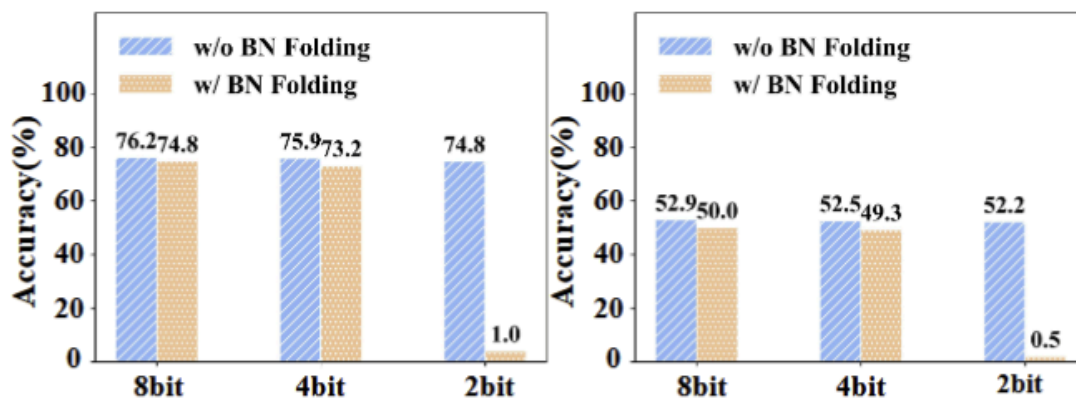


Figure 2: Accuracy on CIFAR100 with ResNet19 and on TinyImageNet with VGG16 under quantization.

Almost all values fall in same quantisation bin since after BN, the values are highly centered around 0. This results in many gradient updates becoming ineffective, thereby severely hampering model convergence

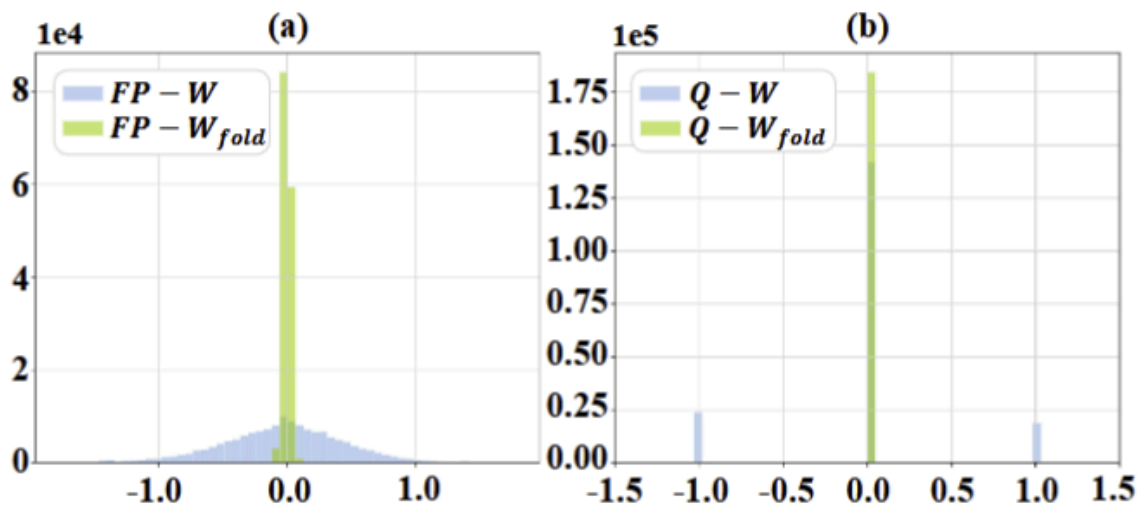


Figure 3: (a) Distribution of raw weights and BN-folded weights before quantization. (b) Distribution of raw weights and BN-folded weights after 2-bit quantization.

Batch wise sigma and mean calculated

$$y = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Divide by $\sigma^2 + \epsilon$ and then round using Pow2STE operation. It quantises the value but keeps it smooth thereby allowing back propagation.

“Pow2STE is a technique that approximates a scaling factor to the nearest power-of-two for efficient hardware implementation, while using a straight-through estimator to allow gradients to pass through during training”

It works by using the actual non-differentiable function during the forward pass, but substituting it with an identity function (or another surrogate) during backpropagation, allowing gradients to pass directly through.

Performance

State-of-the-art accuracy and memory efficiency across CIFAR-10/100, TinyImageNet, ImageNet-1k, and DVS-CIFAR10.

Hardware

FPGA deployment: 15× lower latency, 3.6× higher throughput, 2.2× better energy efficiency vs. competing accelerators.

Experimental Results: Accuracy & Memory

Evaluated on CIFAR-10, CIFAR-100, TinyImageNet, ImageNet-1k, DVS-CIFAR10

95.53%

CIFAR-10 (4w-4u)
vs 90.79% prior SOTA

75.35%

CIFAR-100 (4w-4u)
vs 74.78% prior SOTA

52.07%

TinyImageNet (4w-4u)
vs 51.78% prior SOTA

10.9×

Memory reduction
vs weight-only quant on
ImageNet

Key Findings

Dataset / Config	HardF-SNN Acc.	vs. Best Competitor	Memory (MB)
CIFAR-10 2w-2u	94.49%	+1.60% vs Hasssan (2024)	68.28
CIFAR-100 4w-4u	75.35%	+0.62% vs Wei (2025b)	114.82
TinyImageNet 2w-2u	51.87%	+0.20% vs Wei (2025b)	54.73
DVS-CIFAR10 8w-8u	79.53%	+1.59% vs Hasssan (2024)	80.48

Page 7 / 9

Hardware Performance: FPGA Deployment Results

HardF-SNN deployed on a dedicated FPGA accelerator (xc7z035) — compared against SyncNN, E3NE, FireFly, and SiBrain

CIFAR-10 — VGG7

Method (Bit)	Acc.%	Latency (ms)	GOPS
SyncNN (4bit)	90.37	16.13	28.5
FireFly (8bit)	91.36	4.16	273.1
SiBrain (8bit)	90.25	18.90	130.8
Ours (8bit)	91.91	3.99	284.74
Ours (4bit)	91.83	2.22	511.75
Ours (2bit)	90.68	1.23	923.66

CIFAR-100 — VGG11

Method (Bit)	Acc.%	Latency (ms)	GOPS/W
E3NE (6bit)	65.00	163.90	0.72
FireFly (8bit)	64.28	8.52	107.9
SiBrain (8bit)	66.97	19.00	83.4
Ours (8bit)	67.26	7.88	79.74
Ours (4bit)	66.55	4.23	139.92
Ours (2bit)	65.49	2.36	241.06

★ 15× latency reduction vs SiBrain (CIFAR-10) ★ 3.6× throughput improvement vs FireFly (CIFAR-100)
efficiency vs FireFly (CIFAR-100) ★ 4-bit: optimal accuracy/efficiency balance

★ 2.2× energy

Page 8 / 9

Spikes to real value by decoder and back to spikes by encoder